

UACR validations: which logic each project follows

Ionic: health-worker-app, uacr-scan-card page (accurate values). RN: helloKidney_FE, TimerCameraUploader (inaccurate values). Each validation has an ID. Reply with the IDs you want changed, for example "change V4 and V8".

The main finding

V4, the framing check, uses different logic. Ionic looks for the printed ArUco corner markers on the card and needs at least 3 of 4. The RN port replaced this with a generic edge-box guess because the marker detector was left out. The server reads the strip colour against those printed markers and reference squares, so a photo that passes in RN without clear markers can give a wrong value. This is my reading of the code. I have not run either app, so it is unproven.

Summary of verdicts

Verdict	Count	Validations
DIFFERENT LOGIC	3	V1, V4, V8
SAME LOGIC, DIFFERENT INPUT	4	V2, V3, V5, V10
SAME LOGIC	1	V9
OFF IN BOTH	2	V6, V7

The threshold numbers and the maths for sharpness, exposure and contrast are identical in both projects. What differs is the image each check reads and, for framing, the whole method.

Validation by validation

ID	Validation	Ionic app logic (accurate)	This RN app logic	Verdict	Why it matters
V1	QR / card detected	Live: jsQR on each preview sample. Tries a top-centre crop first, then the full frame, with normal and inverted colours. Once a QR is read it stays passed for the session. After capture: the QR text is checked against the backend (resolve-qr result must equal the uacr_encrypted secret). Failure means retake.	expo-camera native barcode scanner. The first QR read locks the state. No backend check. The raw QR text is sent to the server as qr_code.	DIFFERENT LOGIC	RN never confirms the card is a genuine card. Ionic never sends the QR text to prediction.
V2	Sharpness	Laplacian (4 neighbours) on greyscale, nearest-neighbour downsample to 800 px. Pass when variance is above 200.	Same code, ported line for line. Threshold 200.	SAME LOGIC, DIFFERENT INPUT	Live: Ionic reads a quality-100 preview frame. RN reads a quality-0.3 still. After capture: Ionic reads the full photo. RN reads the cropped strip.
V3	Exposure	Greyscale histogram at 400 px. Dark below 85, mid 85 to 169, bright 170 and up. Pass when dark under 55 percent, bright under 80 percent, mid over 5 percent.	Same code and thresholds.	SAME LOGIC, DIFFERENT INPUT	Same input differences as V2.
V4	Bounds / framing (card fully in view)	ArUco marker detection with js-aruco2, dictionary ARUCO_MIP_36h12. Markers under 400 square px are ignored. The card has 8 markers. Each marker is placed in a quadrant relative to the marker centroid and the one farthest from centre is kept, giving up to 4 corners. Pass when at least 3 of 4 corner markers are found. Messages say how many were found.	Generic edge heuristic. Gradient above 45 on a 500 px greyscale copy. Takes the box around all edge pixels. Pass when that box is at least 15 percent of the frame and touches at most one side, or fills at least 80 percent. Never looks for the printed markers.	DIFFERENT LOGIC	Biggest gap. Ionic only accepts a photo where the printed markers are visible. RN accepts any edgy scene of the right size, so a poorly framed or partly cut-off card can pass.
V5	Contrast	Standard deviation of greyscale at 400 px. Score is deviation divided by 127.5, times 100. Pass at 30 or above.	Same code and threshold.	SAME LOGIC, DIFFERENT INPUT	Same input differences as V2.
V6	Resolution	Function exists but always returns pass. Not used in the gate.	Never built.	OFF IN BOTH	No effect in either app.
V7	Skew / tilt	Two versions exist (edge angle histogram and ArUco corner angle). Neither is in the gate. Device-tilt sensing was removed.	Never built.	OFF IN BOTH	No effect in either app.
V8	Streak needed for auto-capture	Every 200 ms. All checks must pass 8 times in a row (1.6 s). Any failure resets the count to zero. User can also tap.	Every 900 ms. All checks must pass 2 times in a row (about 1.8 s). A failure only subtracts 1. User can also tap.	DIFFERENT LOGIC	RN fires on fewer, coarser samples and forgives failures. Ionic waits for a steadier hold.
V9	Order and stop rule	Sharpness, exposure, bounds, contrast. Stops at the first failure and shows that message.	Same order. After capture RN runs all four but shows the first failure message.	SAME LOGIC	Same visible result.
V10	How pixels are decoded	Browser canvas getImageData.	jpeg-js decoding in JavaScript.	SAME LOGIC, DIFFERENT INPUT	Two decoders can differ by small pixel amounts. Thresholds sit close enough that a few borderline photos could pass in one app and fail in the other.

Suggested order of change

Rank	IDs	What to change	Notes
1	V4	Add the same ArUco check to RN. Ionic uses the js-aruco2 library, which is plain JavaScript and works on decoded pixel data. RN already decodes pixels with jpeg-js, so it can be fed the same way. Use dictionary ARUCO_MIP_36h12, ignore markers under 400 square px, keep the farthest marker per quadrant, pass at 3 of 4.	Would replace the Sobel guess in both the live loop and the post-capture gate. Adds one dependency.
2	V2, V3, V5, V10	Run these checks on the same image Ionic does: the full photo after capture, and a higher quality live frame.	Low risk. Affects which photos are accepted.
3	V8	Match the streak: sample faster and require more clean samples in a row, with a full reset on failure.	Live samples in RN are real stills, so going much faster may not be possible.
4	V1	Add the backend QR validation Ionic does, and stop sending qr_code if the server does not need it.	Needs the backend owner to confirm the field.
5	A11 to A13	Send the image through the same .NET endpoint as Ionic.	Only worth doing if the validation changes above do not close the gap.

How to test fairly

- Use the same physical card, table, light and distance in both apps.
- Run each app three times and note creatinine, microalbumin and uACR.
- Also note whether RN accepts photos where a card corner is hidden. Ionic would reject those.

Appendix: capture and upload differences

These sit outside the validations but also change the image or request the server receives. Areas A4, A8 and A9 were aligned with Ionic last round.

ID	Area	Ionic app (accurate)	This RN app (now)
A1	Camera engine	capacitor-community camera-preview 8.0.1 (Android Camera1 API).	expo-camera 17.0.10 (CameraX / AVCapturePhotoOutput).
A2	Live frames	Preview samples (captureSample, quality 100) every 200 ms. No still taken.	A real still (quality 0.3) every 900 ms.
A3	Shutter	Auto-capture after the streak, plus a tap button.	Auto-capture after the streak, plus a tap button. (My earlier note said Ionic was tap-only. That was wrong.)
A4	Wait before final capture	350 ms.	350 ms (added last round).
A5	Capture size	Screen pixel size. Plugin picks the closest supported size.	Highest sensor size. No size set.
A6	Capture JPEG quality	70	0.7
A7	Photo processing	Camera JPEG used directly on Android.	skipProcessing false: expo-camera decodes, rotates and re-encodes (one extra JPEG generation).
A8	Strip crop size	About strip size times pixel ratio (about 600 px wide).	Downscaled to the same size last round. Was 2 to 4 times larger.
A9	Crop JPEG quality	0.95	0.95 (was 0.92).
A10	Colour enhancement	None.	None.
A11	Upload format	JSON with image_data as a data URL.	Multipart form with a JPEG file.
A12	Endpoint	api.hellokidney.ai/api/UACRPrediction/predict-uacr (.NET API).	uacrapl.hellokidney.ai/users/process-test (direct).
A13	Request fields	patient_id, test_id, token.	email_id, qr_code, role, hw_id, patient_id.

Where the code lives

Item	Location
Ionic quality checks	health-worker-app/src/app/services/image-quality-validator.service.ts
Ionic ArUco framing check	health-worker-app/src/app/services/aruco-detector.service.ts, validateBoundsViaAruco()
Ionic live loop and streak	health-worker-app/src/app/services/frame-processor.service.ts
Ionic capture, crop, upload	health-worker-app/src/app/pages/neo-docs/uacr-scan-card/uacr-scan-card.page.ts
RN quality checks	helloKidney_FE/app/components/TimerCameraUploader/imageQuality.ts
RN live loop, capture, upload	helloKidney_FE/app/components/TimerCameraUploader/index.tsx